

NCCIA UDC Preparation — Project Guide

A complete MCQ preparation platform for the NCCIA Upper Division Clerk (UDC) job test: a quiz engine (Practice, Exam, Random, and Subject test modes), a dashboard, an admin panel, and an MCQ ingestion pipeline that validates, filters for syllabus relevance, deduplicates, parses correct answers, and saves new questions.

Read this first

This project was built in an environment with no network access. Two things follow directly from that, stated plainly rather than hidden:

1. **The 15 requested scraper sources (TestPoint, GoTest, PakMcqs, PakMCQs.org, Ilmkidunya, TayyarHo, Pak Job MCQs, TestPointsPK, Talib.pk, TestPointPK, JobsAlert.pk, Parho Pakistan, Educated.pk, PaperPK, StudyInfo.pk) ship as documented, gracefully-failing adapter stubs, not working scrapers.** Their live HTML structure was never inspected (impossible without network access), and the project's own instructions said not to guess selectors. Each adapter has the right interface, the right metadata, and a clear TODO for activating it once you've verified you're allowed to scrape that site. One adapter — Demo — is fully working by design, drawing from a local, clearly labeled pool so the entire pipeline is exercised for real. The practical way to get real content in today is the manual import tool at `/admin/import`, which runs through the identical validation/relevance/duplicate-detection pipeline.
2. **The code was not run end-to-end in this environment** — `npm install` needs the npm registry, which wasn't reachable. Every file is complete, real TypeScript — checked with the TypeScript compiler in syntax-verification mode (no errors found) and reviewed by hand — but you should run the verification commands in `INSTALLATION_GUIDE.md` yourself as the real test.

Full detail on both points, and every other documented trade-off, is in

`PROJECT_DOCUMENTATION.md` at the project root — read that file's section 2 before relying on anything below.

What's included

- Practice / Exam / Random / Subject test modes, with server-graded scoring and a subject-wise breakdown.
- A dashboard: total/new/verified MCQ counts, accuracy, best score, recent activity, weak/strong subjects.

- Duplicate detection: exact (hash-based, catches formatting/case variants) and near (Jaccard + TF-IDF cosine similarity, catches paraphrases) — see `tests/duplicate-detection.test.ts` for worked examples straight from the original spec.
- A configurable syllabus (`/admin/syllabus`) driving a rule-based relevance filter — nothing about "what counts as UDC-relevant" is hard-coded.
- A "job profile" system so the same app can later serve other posts (FIA, ASF, PPSC, etc.) without rebuilding anything — just a new profile and syllabus.
- An admin panel: MCQ CRUD + verification workflow, source management, syllabus editing, extraction history with per-source drill-down, manual import (JSON/CSV), CSV/JSON export.
- Dark/light mode, a responsive sidebar/bottom-nav shell, toasts, empty states, skeleton loaders.

Installing it (short version)

```
npm install
cp .env.example .env
# edit .env — see INSTALLATION_GUIDE.md for how to generate
# ADMIN_PASSWORD_HASH and SESSION_SECRET
npm run db:migrate
npm run db:seed
npm run dev
```

Then open `http://localhost:3000` . Full walkthrough, Windows commands, and troubleshooting: `INSTALLATION_GUIDE.md` .

Deploying it (short version)

```
cp .env.example .env # fill in real ADMIN_EMAIL / ADMIN_PASSWORD_HASH / SESSION_SECRET
docker compose up -d
```

This builds against PostgreSQL, runs migrations and the seed automatically, then starts the app. Put a reverse proxy (Caddy or Nginx) in front for HTTPS. Full detail, plus a managed-services alternative and a security checklist: `DEPLOYMENT_GUIDE.md` .

Configuring the syllabus

Go to `/admin/syllabus` . Each subject has: a name, a comma-separated list of include keywords, an optional comma-separated exclude-keyword list, and a minimum-hit threshold. A question is treated as relevant to a subject once it matches at least that many include keywords and none of

the exclude keywords. Edit freely — the change applies to the next extraction or import run, no code changes or redeploy needed.

Adding a source for real

See `docs/ADDING_A_SCRAPER.md` inside the project. Short version: check `robots.txt` /ToS first, inspect the real page structure, implement the adapter's `run()` method to fetch and parse real MCQs (never fabricate one), then enable it from `/admin/sources` .

Adding another job profile (e.g. FIA, PPSC)

`/admin/job-profiles` → create a new profile → make it active → go to `/admin/syllabus` and add that profile's subjects. The quiz pages, dashboard, and extraction pipeline automatically scope to whichever profile is marked active.

What's not included / left as documented follow-up

- Working scrapers for the 15 named sites (see above).
- A separate Redis/Celery or BullMQ job queue — extraction runs in-process instead; documented upgrade path in `src/lib/extraction-engine.ts` and `DEPLOYMENT_GUIDE.md` .
- A confirmed official NCCIA UDC syllabus — the shipped syllabus is a documented, editable reconstruction (`src/lib/default-syllabus.ts`).
- An optional AI relevance double-check layer — off by default, with a documented slot to add one (`AI_RELEVANCE_ENABLED` in `.env.example`).

This list is also in `PROJECT_DOCUMENTATION.md` §11, alongside everything else about how the project is built.